Multi-Output Gaussian Processes and Coregionalization

A Complete Mathematical Reference

---

1. Why Share a Model Across Multiple Outputs?

In many sequential decision-making problems, you are simultaneously learning about several related quantities. A naive approach treats each output independently, building a separate GP for each. This is wasteful in a precise mathematical sense: if the outputs are correlated — if learning about one tells you something about the others — then independent models throw away information.

The multi-output GP formalizes the idea of **joint inference across tasks**. Rather than maintaining separate posterior distributions for each output, it maintains one joint distribution over all outputs simultaneously, allowing evidence about any one output to update beliefs about all the others via the learned correlation structure.

The value of this is concrete. Suppose you have two tasks f_1 and f_2 that are strongly positively correlated. Task f_1 has been observed at many input points; task f_2 has been observed at only a few. In the independent model, f_2's posterior is wide and uncertain everywhere. In the multi-output model, f_2's posterior near f_1's observations is substantially tighter — f_2 is "borrowing strength" from f_1. This is the formal analog of the intuition that related problems should share information.

---

2. The Linear Model of Coregionalization (LMC)

The **Linear Model of Coregionalization** is the standard framework for constructing valid multi-output kernels. The core idea is to model each task's function $f_t(x)$ as a linear combination of Q shared **latent functions** $u_1(x), \ldots, u_Q(x)$:

$$f_t(x) = \sum_{q=1}^{Q} a_{tq}, u_q(x)$$

where each latent function u_q is an independent GP with kernel k_q, and the coefficients ${a_{tq}}$ form a $T \times Q$ matrix A called the **mixing matrix**.

Under this model, the cross-covariance between tasks t and s at inputs x and x' is:

$$\text{Cov}(f_t(x), f_s(x')) = \sum_{q=1}^{Q} a_{tq} a_{sq}, k_q(x, x') = \left[\sum_{q=1}^{Q} k_q(x, x'), \mathbf{a}_q \mathbf{a}*q^T|*right]{ts}$$

This shows that the LMC kernel factorizes as a sum of Kronecker products between a task covariance structure and an input covariance structure — a factorization with profound computational consequences.

---

3. The Intrinsic Coregionalization Model (ICM)

The simplest special case of the LMC uses $Q = 1$ — a single shared latent function. This is the **Intrinsic Coregionalization Model**:

$$\text{Cov}(f_t(x), f_s(x')) = B_{ts}, k(x, x')$$

where $B = A A^T$ is a $T \times T$ positive semidefinite matrix called the **coregionalization matrix**, and k is a single shared input kernel.

The matrix B is the heart of the ICM. Its diagonal entries B_{tt} are the marginal variances of each task. Its off-diagonal entries B_{ts} encode the inter-task covariances. The **correlation** between tasks t and s is:

$$\rho_{ts} = \frac{B_{ts}}{\sqrt{B_{tt} B_{ss}}} \in [-1, 1]$$

The joint kernel over all tasks is a **Kronecker product**:

$$K_{\text{joint}} = B \otimes K_{\text{input}}$$

where K_{input} is the $n \times n$ kernel matrix over the shared input points, and $\otimes$ denotes the Kronecker product. For a system with T tasks each observed at n points, this matrix has size $(Tn) \times (Tn)$.

---

4. Kronecker Structure and Computational Efficiency

The Kronecker product structure $K_{\text{joint}} = B \otimes K_{\text{input}}$ is not merely a mathematical convenience — it is the key to making multi-output GP inference tractable.

Cholesky decomposition factorizes. For a Kronecker product, the Cholesky decomposition satisfies:

$$\text{chol}(B \otimes K_{\text{input}}) = \text{chol}(B) \otimes \text{chol}

(K_{\text{input}})$$

This means instead of computing one Cholesky decomposition of a $(Tn) \times (Tn)$ matrix at cost $O(T^3n^3)$, you can compute two separate decompositions: one of the $T \times T$ matrix B at cost $O(T^3)$, and one of the $n \times n$ matrix K_{input} at cost $O(n^3)$. The total cost is $O(T^3 + n^3)$ versus $O(T^3 n^3)$ — an exponential improvement.

Matrix-vector products use the vec-trick. For a Kronecker product $M = A \otimes B$, multiplication by a vector $\text{vec}(X)$ satisfies:

$$M, \text{vec}(X) = \text{vec}(BXA^T)$$

where $\text{vec}(\cdot)$ denotes column-wise vectorization. This reduces an $O(T^2n^2)$ matrix-vector product to $O(Tn(T + n))$.

In practice, when observations are unequally distributed across tasks (some tasks have more data than others), the pure Kronecker structure breaks down and the full $(Tn_{\text{total}}) \times (Tn_{\text{total}})$ matrix must be assembled directly. The efficiency gain is recovered approximately, but the full Cholesky factorization is required.

---

5. Joint Posterior Inference

Given observations from all tasks, the joint posterior for a query task t at new input points X_* follows the standard GP conditioning formula, applied to the joint kernel.

Let K_{joint} denote the training kernel matrix over all tasks and training points, $\mathbf{Y}*{|text{joint}}$ the stacked observation vector, and $K*^{(t)}$ the cross-covariance between the query points for task t and all training observations.

The posterior mean for task t is:

$$\boldsymbol{\mu}*^{(t)} = K^{(t)} K_{|text{joint}}^{-1} |mathbf{Y}{|text{joint}} = K^{(t)} \boldsymbol{\alpha}$$

where $\boldsymbol{\alpha} = K_{\text{joint}}^{-1} \mathbf{Y}_{\text{joint}}$ is computed once via two Cholesky solves and shared across all task queries.

The posterior variance is:

$$\boldsymbol{\sigma}*^{(t)} = |text{diag}!|left(B{tt}, K_{\text{input}}(X_, X_) - K_^{(t)} K_{|text{joint}}^{-1} K_^{(t)T}\right)$$

The cross-covariance block $K_^{(t)}$ between the query task t at points $X_$ and training task s at points X_s is:

$$\left[K_^{(t)}|right]{|cdot, |text{block}(s)} = B{ts}, k(X_, X_s)$$

This is the equation that makes cross-task inference work: the covariance between a query for task t and the observations of task s is B_{ts} times the input kernel. When B_{ts} is large, t's posterior is strongly influenced by s's observations. When $B_{ts} = 0$, the tasks are independent and s's observations contribute nothing to t's posterior.

---

6. Joint Thompson Sampling

In the multi-output setting, Thompson Sampling must be generalized to draw **one coherent joint sample** from the full multi-task posterior, rather than independent samples from each task's marginal. The importance of this is subtle but fundamental.

If you draw independent samples for each task, you discard the correlation information. An optimistic sample for task 1 in the independent case tells you nothing about task 2. But in the joint model, if tasks 1 and 2 are positively correlated ($B_{12} > 0$), an optimistic draw for task 1 should tend to produce an optimistic draw for task 2 as well — the GP says these tasks move together, so your uncertainty about one should be correlated with your uncertainty about the other.

The joint sample is drawn from the $(T \cdot G)$-dimensional posterior distribution over all T tasks at all G grid points simultaneously:

$$\tilde{\mathbf{f}} \sim \mathcal{N}(\boldsymbol{\mu}{|text{full}},| |Sigma{\text{post}})$$

where $\boldsymbol{\mu}{|text{full}}$ is the (TG)-vector of posterior means and $|Sigma{\text{post}}$ is the $(TG) \times (TG)$ posterior covariance matrix:

$$\Sigma_{\text{post}} = K_{**}^{\text{full}} - K_^{|text{full}} K_{|text{joint}}^{-1} K_^{\text{full}T}$$

The Cholesky draw proceeds identically to the single-output case: $\tilde{\mathbf{f}} = \boldsymbol{\mu}{|text{full}} + L{\text{post}} \mathbf{z}$ where $L_{\text{post}} = \text{chol}(\Sigma_{\text{post}})$ and $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I)$.

The sample $\tilde{\mathbf{f}}$ is then partitioned into T blocks of G values each, one per task. The recommended intensity for each task is the grid point that maximizes its block of the sample. Arm selection uses these sampled values directly: the arm with the highest sampled value (after applying any stealth or risk adjustment) is selected.

---

7. The Coregionalization Matrix B: Learning and Identifiability

7.1 The Cholesky Parametrization

The matrix B must be positive semidefinite for the joint kernel to be valid. The standard approach is to parametrize B through its Cholesky factor:

$$B = L_B L_B^T$$

where L_B is lower triangular with positive diagonal entries. Any matrix of this form is automatically positive semidefinite, so the optimizer can search over all real-valued entries of L_B without ever generating an invalid B.

7.2 The Rotation Ambiguity and Its Resolution

The parametrization $B = L_B L_B^T$ has a rotation ambiguity: for any orthogonal matrix Q (satisfying $Q^T Q = I$), we have $(L_B Q)(L_B Q)^T = B$. This means infinitely many L_B matrices represent the same B, which causes problems for warm-starting the optimizer — two equivalent representations might be far apart in parameter space, making the gradient landscape misleading.

The resolution is to constrain the diagonal entries of L_B to be strictly positive. This eliminates the orthogonal rotation freedom and selects a unique representative from each equivalence class of L_B matrices that represent the same B. The positive-diagonal constraint is enforced via the log transform: store $\log L_B[i,i]$ as the free parameter and recover $L_B[i,i] = \exp(\cdot)$.

7.3 The Scale Ambiguity

A second non-identifiability arises because multiplying B by a constant c and dividing the input kernel variance by c leaves the product $B \otimes K_{\text{input}}$ unchanged. This means the overall scale of B is not identified separately from the kernel scale.

The standard resolution is to normalize: fix $B_{00} = 1$ (anchoring the first task's variance to unity), or fix $\text{tr}(B) = T$ (so the average task variance is one). Either convention yields identifiable off-diagonal entries, whose interpretation as correlations is then unambiguous.

7.4 Coverage-Based Regularization

When tasks have unequal amounts of data, the off-diagonal entries of B are harder to estimate reliably — there may simply be too few observations of a rarely-evaluated task to determine its correlation with other tasks. In this case, the marginal likelihood optimization

can over-fit the off-diagonal entries, assigning spuriously high correlations based on limited evidence.

The principled solution is a prior that penalizes large off-diagonal entries of L_B in proportion to the coverage imbalance. The coverage imbalance is measured by the coefficient of variation of observation counts:

$$\lambda_{\text{reg}} = \lambda_0 \cdot \frac{\text{std}(n_1, \ldots, n_T)}{\text{mean}(n_1, \ldots, n_T)}$$

The regularized objective becomes:

$$\mathcal{L}(\theta) = -\log p(\mathbf{Y} \mid X, \theta) + \lambda_{\text{reg}} \sum_{i > j} L_B[i,j]^2$$

When all tasks have equal coverage, $\lambda_{\text{reg}} \approx 0$ and the prior is uninformative. When coverage is highly unequal, λ_{reg} is larger and B is pulled toward the identity matrix — toward independence — preventing the model from hallucinating correlations that the data cannot support.

---

8. The Spatio-Temporal Extension

The ICM model assumes the same input kernel $k(x, x')$ applies at all times. For processes that evolve over time, a more appropriate model combines a spatial kernel in the intensity dimension with a temporal kernel:

$$k_{\text{ST}}((x, t), (x', t')) = k_{\text{Matérn}}(x, x') \cdot k_{\text{per}}(t, t')$$

The product structure means the function must be smooth in intensity *and* periodic in time *simultaneously*. The periodic kernel:

$$k_{\text{per}}(t, t') = \exp!\left(-\frac{2\sin^2(\pi|t - t'|/p)}{\ell_t^2}\right)$$

introduces a period p that can be learned from data alongside the other hyperparameters. This allows the model to discover and exploit periodic structure in the reward landscape — for example, if rewards follow cycles correlated with external processes.

In the multi-output context, each task now has observations at $(intensity, time)$ pairs, and the cross-task covariance block between tasks t and s becomes:

$$\left[K_{\text{joint}}\right]*{t,s} = B*{ts} \cdot K_{\text{Matérn}}(X_t^{\text{int}}, X_s^{\text{int}}) \cdot K_{\text{per}}(X_t^{\text{time}}, X_s^{\text{time}})$$

The period p is a shared hyperparameter across all tasks, encoding the belief that

whatever temporal cycle exists in the environment affects all arms similarly.

See also: Gaussian Process Regression; Cholesky Decomposition; Non-Stationary Gaussian Processes; The Two-Timescale Bayesian Update